从零理解并实践神经网络逆运动学

面向 Windows PyCharm 和 Python 3.12.1 的 TIAGo 逆运动学入门教程

这份教程帮助你把“看懂一篇论文”变成“完成一次能够检查结果的实验”。学习顺序是：理解输入和输出，运行一个两关节练习，再把相同流程用于 TIAGo 或 KUKA 的数据。你的第一轮目标是建立完整流程，然后逐步提高精度。

依据论文：Ana Calzada-Garcia 等，Inverse Kinematics for Robotic Manipulators via Deep Neural Networks: Experiments and Results，Applied Sciences，2025，15，7226。DOI：https://doi.org/10.3390/app15137226 。文中的“论文第 N 页”均指本地 PDF 的第 N 页。

学习路线

顺序	学习内容	学完后你能做什么
1	正运动学与逆运动学	说清楚数据如何产生，网络预测什么
2	跑通两关节教学练习	获得数据、模型、训练曲线和末端误差
3	认识论文的数据和 SimpleMlp	检查七关节 CSV，理解网络的输入输出尺寸
4	数据划分与归一化	正确隔离训练、验证和测试数据
5	用正运动学评价模型	区分角度标签误差与末端到达误差
6	迁移到 KUKA	明确模型、坐标、采样和评价需要怎样改动

配套文件放在本教程同一文件夹。**01_warmup_numpy.py** 自动生成两关节数据并训练；**02_train_pose_csv.py** 读取你准备的关节与位姿 CSV；**ik_numpy.py** 是二者共用的教学网络实现。先读前五页并运行练习，再继续读七关节部分。

本教程的代码使用 NumPy，借鉴论文的数据到模型到正运动学验证的流程。它是便于入门的独立实现。作者的正式实现使用 Keras；若要复现论文表格中的数字，还需要匹配作者的模型、数据、预处理、训练设置与评价方法。

配套代码的验证范围

两关节数据生成、训练、保存、重新加载和 FK 评价已实际运行。教程中的七关节 CSV 入口已做格式与处理流程检查，未在这里完成 TIAGo 公开数据训练或 KUKA 仿真复现。教学结果单独保存在 **runs/verified_warmup**，用于帮助你理解输出文件。

当前版本按你已经跑通的 Python 3.12.1 和 NumPy 2.5.4 环境编写。你已完成 **warmup_01**，可以先看第 4 节的结果解释；需要再次练习时，按第 3 节使用新的输出目录。

1 先理解机械臂在学习什么

想象你告诉机械臂：“把手伸到桌面上某个点，并让手掌朝下。”目标位置告诉它到哪里，目标朝向告诉它怎样摆放末端。位置和朝向合在一起，称为位姿。

名词	通俗解释	本教程中的记号
关节角度	每个转动关节转了多少	q，若有 n 个关节，就是 n 个数
末端位置	工具或手爪的位置	p，三维任务一般是 x、y、z
末端朝向	工具朝哪个方向	r，这里可以用四元数表示
正运动学 FK	已知角度，求末端位姿	T = FK(q)
逆运动学 IK	已知目标位姿，求一组可用角度	q_hat = 网络(T_target)

例如，一条两关节机械臂可以用两个角度决定手的位置。数据生成时，我们先选角度，再算位置；训练时，把这一对数据的输入与输出交换。

```
生成数据   关节角度 q  →  正运动学 FK  →  末端位姿 T
训练网络   末端位姿 T  →  神经网络     →  预测角度 q_hat
检查结果   预测角度    →  正运动学 FK  →  检查是否达到目标
```

为什么需要检查末端

同一个目标往往有多组角度解，例如肘部可以朝上或朝下。只把预测角度与某一组标签比较，可能把另一组有效解误判为失败。实际需要检查的是预测角度经过 FK 后的末端位置与朝向，以及关节限位等条件。

这篇论文的任务范围

论文研究固定底座上的 TIAGo 七自由度机械臂，采用静态位姿到角度的映射。论文第 2-4 页明确没有把轨迹或速度作为研究对象。完成这个模型后，你具备了一个角度求解模块；要研究未来实际角度、连续轨迹或避障，还需要相应的数据和实验设计。

动手检查：合上教程，用一句话说出“我的网络输入是什么，输出是什么”。如果答案中出现“下一时刻”，先重新检查你要研究的是逆运动学还是未来状态预测。

2 使用你现有的 Python 3.12 环境

你已经在 PyCharm 成功完成一次训练，现有虚拟环境可以继续使用。下面的版本来自实际检查；环境已选好、依赖已装好的步骤可以直接跳过。

项目	当前设置
操作系统与编辑器	Windows 和 PyCharm
Python	3.12.1 64 位
NumPy	2.5.4，配套 requirements.txt 已固定此版本
虚拟环境	教程项目内的 .venv

在 PyCharm 中选择现有解释器

打开 Python Interpreter 设置，选择现有本地解释器。右下角若显示 **No interpreter**，也可以从那里进入。解释器应指向以下文件：

```
D:\Swansea-杜亦冯\biyeshj\output\tutorials\tiago_ik_beginner\.venv\Scripts\python.exe
```

这一步选择的是 Python 可执行文件。选好后，PyCharm 的状态栏应显示当前 Python 环境；后续运行使用同一个解释器。

确认版本与安装依赖

在 PyCharm 底部 Terminal 中执行下列 PowerShell 命令。直接指定 **.venv** 中的 Python，可以确保安装和运行使用相同环境。

```
Set-Location -LiteralPath 'D:\Swansea-杜亦冯\biyeshj\output\tutorials\tiago_ik_beginner'
.\.venv\Scripts\python.exe --version
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -c "import numpy; print(numpy.__version__)"
```

预期版本分别为 Python 3.12.1 和 NumPy 2.5.4。依赖已经安装时，pip 显示 **Requirement already satisfied** 属于正常结果。

遇到安装项目包的报错

如果看到 **pip install -e** 和“找不到 setup.py 或 pyproject.toml”，说明执行了安装项目包的操作。这套教程通过上面的 **-r requirements.txt** 安装依赖，再直接运行脚本。关闭该报错提示后，继续使用现有解释器即可。

3 在 PyCharm 运行两关节练习

练习中的两根连杆长度分别为 0.6 米和 0.4 米。程序采样两个关节角度，再用 FK 得到末端的二维位置，并学习从位置回到角度。它限制肘关节在同一侧弯曲，帮助初学者先避开混合逆解分支的问题。

```
x = 0.6 cos(q1) + 0.4 cos(q1 + q2)
y = 0.6 sin(q1) + 0.4 sin(q1 + q2)
```

例如 q1 = 0、q2 = π/2 时，末端在 x = 0.6 米、y = 0.4 米。默认采样 6000 条数据，最多训练 200 轮，随机种子为 42。

你已完成的第一次运行

右键 **01_warmup_numpy.py**，选择 Run，会使用默认输出目录 **runs/warmup_01**。你当前已经完成这一步，控制台显示 **Process finished with exit code 0**，结果也已保存。**ik_numpy.py** 是共用模块，练习入口是 **01_warmup_numpy.py**。

再次运行时使用新目录

程序保留已有结果；当前 **warmup_01** 已存在，再次使用默认参数会提示目录已存在。在 PyCharm Terminal 中，确认工作目录是教程项目，再执行：

```
.\.venv\Scripts\python.exe 01_warmup_numpy.py --out runs/warmup_02
```

以后依次换成 **runs/warmup_03**、**runs/warmup_04**。如果希望继续使用 PyCharm 的 Run 按钮，编辑该脚本的运行配置：

设置项	填写内容
Script path 脚本	项目内的 01_warmup_numpy.py
Python interpreter 解释器	项目内 .venv 的 Python 3.12.1
Working directory 工作目录	教程项目 tiago_ik_beginner 文件夹
Script parameters 或 Parameters 参数	--out runs/warmup_02

运行后，结果应出现在你指定的新目录中。出现 **epoch 20: train=..., val=...** 说明正在训练；训练时间随电脑而变化。运行结束后，打开新目录中的 **training_curve.svg** 和两个指标 JSON 文件。

你原来的结果仍在 **runs/warmup_01**。下面一节仍用这个已完成的实验讲解结果。两关节结果属于教学机械臂，不能作为 TIAGo 或 KUKA 的实验结果。

4 看懂第一次运行的结果

训练结束后，先看下表中的三个文件，不必马上阅读网络内部的所有代码。

文件	你要看什么
training_curve.svg	用浏览器打开；看训练误差和验证误差的变化
metrics.json	查看关节角度误差，及其是否优于简单基线
warmup_fk_metrics.json	查看预测角度经过 FK 后的末端位置误差
model.npz	网络权重和归一化参数，供以后加载预测
test_predictions.csv	每个测试目标、标签角度和预测角度
split_rows.npz	训练、验证、测试分别用了哪些原始数据行

你已在 Python 3.12.1 和 NumPy 2.5.4 下完成 6000 条样本、200 轮、seed = 42 的运行：平均末端位置误差约 4.61 毫米，95 分位约 10.49 毫米，约 93.7% 的测试目标误差小于 10 毫米。改变采样量、种子或环境后，结果可能变化。10 毫米只是教学中预先选定的阈值。

为什么同时看训练和验证曲线

训练集用于调整权重；验证集用于判断训练效果和选模型。训练误差下降而验证误差持续上升，常见原因是模型开始记住训练数据，或者训练数据与验证数据差异很大。

测试集留到模型设置确定后再评价。如果看过测试结果后反复改模型，原来的测试集就参与了调参，需要另准备最终测试数据。

基线是什么

本练习还计算了“无论输入是什么，都输出训练集平均角度”的误差。它只是一个简单参照，用来判断网络有没有学到关系。用于正式机械臂实验时，还应加入传统数值 IK 等更有意义的基线。

试着读取一个预测结果

```
import numpy as np
from ik_numpy import load_prediction

goal = np.array([[0.6, 0.4]])
angles = load_prediction("runs/warmup_01/model.npz", goal)
print(angles)  # 两个预测角度，单位为弧度
```

这段预测示例只适用于两关节练习生成的模型。七关节模型需要与训练时完全一致的位姿维度、顺序和坐标系。

5 认识论文中的七关节数据

论文第 4-5 页说明两种生成方法：在 Gazebo 中运动并读出状态，或者随机采样关节角度并通过 FK 计算位姿。第一种包含模拟运动的状态读取，第二种更快；两种都需要核对机器人模型、参考系和数据质量。

字段	含义	数量	本教程采用的单位或约定
arm_1 至 arm_7	七个关节角度	7	弧度 rad
ee_x ee_y ee_z	末端位置	3	米 m；相对于约定的参考系
ee_quat_x ee_quat_y ee_quat_z ee_quat_w	末端朝向	4	单位四元数；列顺序为 x y z w

作者代码的 CSV 表头使用上面这些名字。一行是同一组角度与其对应的末端位姿，共 14 个数。训练时，从中取 7 个位姿值作为 X，取 7 个角度值作为 Y。

```
X.shape = (N, 7)   每行 3 个位置值和 4 个四元数值
Y.shape = (N, 7)   每行 7 个关节角度
```

N 是样本数。“7 个输入”与“7 个关节输出”恰好数目相同，物理含义完全不同。若 KUKA 实际有六个关节，则输出应有 6 列。

先检查数据再训练

打开 CSV 前几行，确认没有缺失值或文本混在数值列中。角度 90 度应转换成 π/2，约 1.5708 rad；100 毫米应转换成 0.1 米。

四元数包含四个数字，不是四个转角。其平方和应接近 1。**r** 与 **-r** 表示同一个朝向。固定表示顺序，并用一个已知姿态核对导出值；某些库返回 w x y z，不能仅凭列名认为它已符合本教程约定。需要统一符号时也应保持训练与推理一致，并认识到符号规则的边界可能出现不连续。

检查命令如下。将 **data/tiago.csv** 换成实际 CSV 路径：

```
.\.venv\Scripts\python.exe 02_train_pose_csv.py --csv data/tiago.csv --check-only
```

输出应说明位姿 X 和角度 Y 的形状。配套读取器支持二维位置、三维位置或三维位置加四元数；旋转矩阵数据需要另行扩展读取器。

6 划分数据并做归一化

论文第 6 页介绍四种缩放方法，并描述训练、验证、测试为 70%、20%、10%。教学代码采用这个比例。例如 6000 条独立样本对应 4200 条训练、1200 条验证、600 条测试。

先划分数据，再从训练集计算缩放参数，最后把相同参数用于验证、测试和以后输入的目标。这样评价时不会提前使用测试数据的统计信息。

为什么要缩放

位置值、角度值和四元数值的分布不同。缩放有助于训练，使不同列具有更合适的数值尺度。缩放后的数值暂时不再是可以直接下发给机械臂的物理角度。

教学代码采用论文比较过的 robust scaling，中文可以理解为基于中位数和四分位距的缩放：

```
缩放值 = (原始值 - 训练集中位数) / 训练集四分位距
原始值 = 缩放值 × 训练集四分位距 + 训练集中位数
```

四分位距是第 75 百分位减第 25 百分位。例如训练集某个角度的中位数为 1.2 rad、四分位距为 0.8 rad，则 1.6 rad 缩放后为 0.5；预测值 0.5 还原后就是 1.6 rad。

代码实际做了什么

```
train_end = int(N * 0.7)
val_end = int(N * 0.9)
# 使用打乱后的索引切分，三个集合互不重叠
center, scale = fit_robust(X_train)
X_train_scaled = (X_train - center) / scale
X_val_scaled = (X_val - center) / scale
X_test_scaled = (X_test - center) / scale
```

输出角度 Y 也有自己的一套缩放参数。代码把输入、输出缩放参数和网络一起保存；推理时必须使用这套参数，不能对一个新目标重新计算中位数。

冗余多解需要额外处理

精确重复输入可以删除，但删除重复行并不能解决冗余机械臂的一对多映射。非常相近的目标仍可能对应差异很大的角度。若出现角度平均、末端误差大或相邻输出跳变，可以限定采样区域、统一逆解选择规则，或在后续研究中加入当前关节配置。

本教程的随机划分适用于独立静态采样。若 CSV 来自连续轨迹，应按完整轨迹或场景划分，不能把相邻帧随机分到训练和测试中。

7 理解并训练第一个网络

论文第 7 页的 SimpleMlp 很适合作为起点：7 个输入，160 个隐藏神经元，7 个输出。隐藏层使用 ReLU，原论文输出层使用 tanh。更深并不自动更好，第一轮先掌握一个模型。

```
位姿输入    →    160 个隐藏神经元    →    关节角度输出
7 个数          ReLU                  7 个数
```

“160 个神经元”表示网络中间有 160 个计算单元，不表示机械臂有 160 个关节。权重是训练过程中调整的数字。ReLU 使模型能够表达非线性关系，否则多层线性运算仍相当于一个线性模型。

四个训练名词

名词	你可以怎样理解	教学代码设置
Loss 损失	预测角度和标签差多少	缩放角度的均方误差 MSE
Batch 批次	每次用多少样本调整权重	最多 128 条
Epoch 训练轮次	完整遍历一遍训练样本	最多 200 轮
Learning rate 学习率	每次调整权重的步幅参数	0.001，使用 Adam

教学程序保存验证损失最低的权重。如果连续 25 轮没有改善，会提前停止。测试集不会用于更新权重或选择训练轮次。

教学实现与原论文的区别

本教程保留一个 160 单元的 ReLU 隐藏层，用 NumPy 实现前向计算与 Adam 更新。输出使用线性层，使 robust scaling 后可能超出 [-1, 1] 的标签仍可表达。原论文 SimpleMlp 的输出是 tanh，其输出范围为 [-1, 1]；若要按原设置复现，必须同时核查输出缩放范围。线性层本身不保证预测角度满足物理限位。

准备好真实静态 CSV 后运行：

```
.\.venv\Scripts\python.exe 02_train_pose_csv.py --csv data/tiago.csv --out runs/tiago_01
```

把文件换成符合表头约定的 KUKA 数据也能进入相同训练流程。输入输出尺寸会从 CSV 中读取。每次实验使用新目录，并保留数据版本、种子、缩放参数及模型。训练脚本输出的是角度标签误差；真实机械臂的 FK 验证需要按下一页完成。

8 用正运动学判断预测有没有用

先还原预测角度的单位，再调用与数据生成一致的 FK。比较的是目标末端与预测末端，不只是某组角度标签。

测试目标 T_target → 缩放 → 网络 → 缩放角度
缩放角度 → 逆缩放 → 物理角度 q_hat → FK → 预测末端 T_hat
最后比较 T_hat 与 T_target

指标	含义	建议报告方式
关节 MAE	预测角度与选定标签的平均绝对差	各关节分别报告 rad 或度，作为辅助
末端位置误差	预测位置与目标位置的欧氏距离	均值、中位数、95 分位，单位 mm
朝向误差	预测朝向与目标朝向的差异	明确公式；角度误差可报告度
有效解比例	同时满足预设任务条件的比例	写清位置、朝向和限位条件
求解时间	获得结果所需时间	记录硬件、批量大小、计时范围

例如目标位置为 (0.30, 0.20, 0.50) 米，预测末端为 (0.31, 0.20, 0.50) 米，则位置误差为 0.01 米，也就是 10 毫米。

朝向误差要明确公式

论文实际采用四元数距离 **min(||r1-r2||, ||r1+r2||)**。单位四元数下，它的范围为 [0, sqrt(2)]，是无量纲距离。虽然正文部分将它描述为 radians，也不能直接把数值当作物理旋转角。

如果你希望报告直观的旋转角，可以额外采用下面的公式。两个四元数都应先归一化，并采用一致的分量顺序。

```
dot = abs(np.sum(r_target * r_pred))
angle_rad = 2 * np.arccos(np.clip(dot, 0.0, 1.0))
angle_deg = np.degrees(angle_rad)
```

两关节练习已经自动做了位置 FK 评价。七关节 CSV 训练器没有内置 TIAGo 或 KUKA 的几何模型，因此不会自动给出七关节末端误差。要完成正式评价，需要将 **test_predictions.csv** 中的预测角度送入该机器人的 FK 或仿真模型，再读出位姿。

举例而言，你可以预先选 10 mm、5 度并要求所有关节满足限位，定义实验的有效解。阈值应按自己的任务确定，不应因为测试结果差就随意放宽。如果研究碰撞，还应另外检查自碰撞、环境碰撞以及连接过程。

9 接上 TIAGo 数据与作者代码

作者提供了公开代码和数据入口。先获得一个可以检查字段、单位、坐标系和采样方式的原始 CSV，再使用教学读取器。数据入口见最后一页的来源清单。教程制作时，Zenodo 接口返回访问限制，因此这里不假定它当前的压缩包文件名或内容结构。

作者仓库中最值得先看的文件

文件	作用
tiago_data_generation/pykin_generation.py	采样角度，通过 FK 生成 CSV
tiago_data_generation/pykin_preprocessing.py	选择列、缩放并划分数据
tiago_dnn_mlp/simple_mlp_train.py	定义 SimpleMlp 网络
tiago_dnn_mlp/mlp_config.yaml	网络和训练参数
training.py	训练入口及回调
custom_metrics.py	通过 FK 计算末端误差

如果暂时只有处理后的 **.npy** 文件，应结合相应的 **data_stats.yaml** 理解列顺序、缩放方法和还原方式。教学脚本接收原始物理量 CSV，不能把未知缩放的 **.npy** 直接当成原始数据。

按作者环境生成数据

下面是作者仓库的命令形式，适用于其 Linux 或 Docker 环境。它与本教程已经跑通的 Python 3.12 NumPy 环境是两条独立路线，作者指定的 TensorFlow 和 Docker 设置需要另外配置。作者环境还需要 TIAGo URDF 和 meshes，仓库 README 描述了准备位置。

```
python3 tiago_data_generation/pykin_generation.py --num=10000 --file=beginner --orient=quaternion
python3 tiago_data_generation/pykin_preprocessing.py --file=beginner --norm=3 --name=beginner --curr=1
python3 training.py --name SimpleMlp
```

训练前还需把 **mlp_config.yaml** 中的 **simple_mlp.data_dir** 设置为 **pykin/beginner**。预处理脚本会重建同名输出目录，先备份已有实验。Docker 文件包含特定 TensorFlow、GPU 和资源路径，可能需要按实际环境调整。

第一轮不必把全部作者环境一起装完。先用已运行的两关节练习理解每个文件的作用，再准备 TIAGo CSV。若你的目标直接是 KUKA，也可以通过下一页的 CoppeliaSim 采样流程生成自己的数据，继续使用教学训练入口。

10 把流程迁移到 KUKA

迁移时可以保留监督学习流程和 SimpleMlp 思路。机器人几何、关节数量、限位、末端参考系和训练数据需要匹配你实际使用的 KUKA 型号，不能把 TIAGo 的角度标签直接用来控制 KUKA。

开始采样前写清楚这些信息

信息	需要记录什么
机械臂	型号、实际关节数、关节顺序
角度	弧度或度、零位、正方向、上下限
位姿	基座参考系、工具末端参考系、长度单位
朝向	使用四元数，分量顺序和符号约定
数据	工作空间、采样方式、随机种子、筛选条件

在 CoppeliaSim 中生成静态配对数据

1. 检查模型，用几组已知角度确认运动方向和末端读数。

2. 在关节限位内采样一组目标角度；开始时可以限定较小范围。

3. 若采用动态位置控制，等待关节稳定，再在同一个仿真时刻读取实际角度与末端位姿。

4. 保存“实际角度与对应实际位姿”。若使用理想 FK 直接生成数据，则明确说明它是几何模型数据。

5. 按实验定义剔除异常、无效或碰撞配置，并记录保留下来的采样范围。

6. 先生成建议的 1000 条小数据，检查格式和单位，再考虑扩大数据量。

目标角度与实际角度可能因为控制误差而不同。保存标签时，要确保角度和位姿是同一状态，不能把一个目标指令和尚未到达该目标时的末端位置错误配对。

验证一组网络预测

选一个未参与训练的测试目标，输入网络得到角度，核查限位，再通过相同 FK 或仿真模型计算末端位姿。几何验证可以使用不执行过渡运动的配置检查；若通过动态控制移动到目标，则还要区分几何误差与跟踪误差。

最终输出建议包括：目标位置、预测角度、到达位置、位置误差、朝向误差和是否满足条件。若出现问题，先检查角度顺序、单位、基座与工具参考系，再检查网络。运动到目标的整段路径是否碰撞，是额外的规划和验证问题。

11 做一个自己的研究实验

当数据、网络和 FK 验证都能工作后，先选择一个变量。一个适合你起步的问题是：“训练数据增加后，KUKA 逆运动学预测效果如何变化？”

先固定独立的验证集和测试集，从同一个训练池选取例如 1000、5000、10000 条数据。保持模型和训练规则一致。分别只从各自训练子集拟合缩放参数，记录这一规则；使用多个随机种子，报告均值和波动。数据量是建议设置，不是论文指定要求。

实验	训练样本数	测试位置误差 mm	测试朝向误差 度	有效解比例
A	1000	待实验	待实验	待实验
B	5000	待实验	待实验	待实验
C	10000	待实验	待实验	待实验

除了深度学习，还可以加入一个数值 IK 方法进行比较。统一机械臂、测试目标、限位、误差阈值和硬件；说明数值方法的初值、迭代预算，以及网络是否使用批量推理。数值方法输出与标签不同，并不必然表示求解失败。

常见问题排查

现象	先检查什么
右下角显示 No interpreter	选择项目 .venv/Scripts/python.exe 作为现有解释器
提示没有 numpy	用 .venv 中的 Python 执行 -m pip install -r requirements.txt
找不到 setup.py 或 pyproject.toml	使用 requirements.txt 安装依赖，再直接运行脚本
输出目录已存在	换一个 --out，避免覆盖旧实验
缺失字段或四元数不完整	核对 CSV 表头和导出格式
验证误差明显高于训练误差	数据划分、覆盖范围、过拟合与缩放是否一致
角度误差不大但末端误差大	关节顺序、单位、工具坐标和不同关节的影响
网络输出稳定却到不了目标	多解标签平均、目标超出训练范围、模型精度不足
越界或动作跳变	限位检查、逆解分支、采样范围和连续性

优先把问题定位到数据、模型或验证环节中的某一个，再改参数。一次更改一个主要因素，保留前一次结果，才容易解释为什么效果变化。

12 对照原论文时需要读准的地方

这页用于你完成第一次练习之后，帮助你写出准确的文献笔记。论文适合作为实验框架参考；具体数字和代码版本仍需核对。

位置	已核实内容	你应怎样使用
第 7 页 图 4	SimpleMlp 为一个 160 单元的 ReLU 隐藏层，tanh 输出	可作为简单基线；不要直接跳到 80 层模型
第 16 页 表 3	作者实验中 robust scaling 的末端指标优于列出的其他缩放方式	是该实验的观察，不代表所有机器人都如此
第 18 页 表 4	SimpleMlp 位置误差 0.1395 m，即约 139.5 mm	原论文并未由这个数字证明毫米级定位
第 14 页 式 5	使用四元数弦距离，范围 [0, sqrt(2)]	是无量纲距离；不能直接当作旋转角
第 19-20 页	论文测试的课程学习策略只带来有限变化	第一轮可只使用一个训练集

论文文字与公开代码并非完全一致

已核对作者 v1.0.0 标签。论文第 11 页写 Adam，公开训练入口在 **fit** 前使用 RMSprop，最终评价前才重新编译为 Adam。论文写 70/20/10 划分；该版预处理代码先取 10%，再从余下数据取 20%，实际为约 72/18/10，且 **x_test** 在训练期间用于验证，**x_val** 用于最后评价。判断用途应看实际调用，不能只看文件名。

该版代码还先用全量数据计算缩放统计量，再划分数据。教学代码改为先划分、只拟合训练集统计量，并用明确的 train、validation、test 命名。这些改动意味着教学实现的结果不能直接宣称为严格复现原论文。

哪些地方需要你自己证明

网络总能输出一组数字，并不等于总能输出满足误差阈值、关节限位和碰撞要求的解。论文与传统 IK 的比较使用特定初值，作者也说明未做初值敏感性分析。自己的结论应来自相同测试条件下的任务误差和成功率。

第一轮完成后，你应该能够回答：数据来自哪里；输入输出各是什么；测试数据是否独立；缩放怎样还原；末端误差怎样算；模型在哪些目标上失败。能够回答这些问题，再扩大网络或加入避障。

13 阅读来源与完成清单

建议阅读顺序

目的	原论文位置
理解任务与机器人	第 2-4 页
理解数据生成和预处理	第 4-6 页
理解第一个网络	第 7 页 SimpleMlp 与图 4
理解训练和误差	第 11 页、第 13-15 页
学会读结果	第 16-20 页
了解传统方法对照的条件	第 21 页

可核对的来源

1. 论文 DOI：https://doi.org/10.3390/app15137226

2. 本地论文：D:/Swansea-杜亦冯/biyeshj/推荐论文/1_TIAGo深度学习逆运动学_2025.pdf

3. 作者代码 v1.0.0：https://github.com/anacg1620/tiago_dnn_ik/releases/tag/v1.0.0

4. 作者公开数据入口：https://doi.org/10.5281/zenodo.15365939

5. TensorFlow 官方安装说明：https://www.tensorflow.org/install/pip

6. Keras Dense 层说明：https://keras.io/api/layers/core_layers/dense/

当前 Windows 和 PyCharm 教学路线使用已验证的 Python 3.12.1、NumPy 2.5.4。正式使用作者 Keras 实现时，先阅读其 Dockerfile 和 README，并单独准备对应环境；不要直接假定其旧版依赖适用于当前 .venv。

完成清单

- 能用自己的话解释 FK 与 IK。
- 跑通两关节练习，并打开训练曲线。
- 找到训练、验证、测试三个集合，说明各自用途。
- 解释预测角度为什么需要逆缩放。
- 用 FK 算出位置误差，并能把米转换为毫米。
- 检查七关节 CSV 的单位、列顺序和四元数约定。
- 确定 KUKA 型号、关节数、基座和工具参考系。
- 生成小规模 KUKA 配对数据，并完成一次独立测试。